Multi module visual position application

Multi module visual position application
- 1. Introduction
  - 1.1 What is "Multimodal Visual Localization"?
  - 1.2 Overview of Implementation Principles
- 2. Code Analysis
  - Key Code
    - 1. Tools Layer Entry (`largemodel/utils/tools_manager.py`)
    - 2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)
  - Code Analysis
- 3. Practical Operations
  - 3.1 Configuring the Offline Large Model
    - 3.1.1 Configuring the LLM Platform (yahboom.yaml)
    - 3.1.2 Configuring the Model Interface (`large_model_interface.yaml`)
  - 3.2 Starting and Testing the Feature (Text Mode)
- 4. Common Problems and Solutions
    - Problem 1: "Image file not found" error message

---

1. Introduction

1.1 What is "Multimodal Visual Localization"?

Multimodal visual localization is a technology that combines multiple sensor inputs (such as cameras, depth sensors, and IMUs) with algorithmic processing techniques to accurately identify and track the position and posture of a device or user in an environment. This technology does not rely solely on a single type of sensor data, but instead integrates information from different perception modalities, thereby improving localization accuracy and robustness.

1.2 Overview of Implementation Principles

1. **Cross-modal Representation Learning**: In order for LLMs to process visual information, a mechanism must be developed to transform visual signals into a form that the model can understand. This may involve extracting features using convolutional neural networks (CNNs) or other architectures suitable for image processing and mapping them into the same embedding space as text.
2. **Joint Training**: By designing an appropriate loss function, text and visual data can be trained simultaneously within the same framework, allowing the model to learn to relate these two modalities. For example, in a question-answering system, an answer can be given based on both a text question and the associated image content. 3. **Visually Guided Language Generation/Understanding**: Once effective cross-modal representations are established, visual information can be leveraged to enhance the capabilities of language models. For example, given a photo, the model can not only describe what is happening in the image, but also answer specific questions about the scene and even execute instructions based on visual cues (such as navigating to a location).

---

2. Code Analysis

Key Code

1. Tools Layer Entry (`largemodel/utils/tools_manager.py`)

The `visual_positioning` function in this file defines the execution flow of the tool, specifically how it constructs a prompt containing the target object name and formatting requirements.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def visual_positioning(self, args):
        """
        Locate object coordinates in image and save results to MD file.
        定位图像中物体坐标并将结果保存为MD文件。

        :param args: Arguments containing image path and object name.
        :return: Dictionary with file path and coordinate data.
        """
        self.node.get_logger().info(f"Executing visual_positioning() tool with
args: {args}")
        try:
            image_path = args.get("image_path")
            object_name = args.get("object_name")
            # ... (路径回退机制和参数检查)

            # Construct a prompt asking the large model to identify the
coordinates of the specified object. / 构造提示，要求大模型识别指定物品的坐标。
            if self.node.language == 'zh':
                prompt = f"请仔细分析这张图片，用一个个框定位图像每一个{object_name}的位
置..."
            else:
                prompt = f"Please carefully analyze this image and find the
position of all {object_name}..."

            # ... (构建独立的message上下文)

            result = self.node.model_client.infer_with_image(image_path, prompt,
message=message_to_use)

            # ... (处理和解析返回的坐标文本)

            return {
                "file_path": md_file_path,
                "coordinates_content": coordinates_content,
                "explanation_content": explanation_content
            }
        # ... (错误处理)
```

2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file serves as the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py
```

```
class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface. / 统一的图像推理接口。"""
        # ... (准备消息)
        try:
            # 根据 self.llm_platform 的值，决定调用哪个具体实现
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... 调用通义模型的逻辑
                pass
            # ... (其他平台的逻辑)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code Analysis

The core of the visual positioning function lies in **guiding large models to output structured data through precise instructions**. It also follows a layered design with a tool layer and a model interface layer.

1. **Tools layer (`tools_manager.py`)**:
    - The `visual_positioning` function is the core of this functionality. It accepts two key parameters: `image_path` (the image path) and `object_name` (the name of the object to be positioned).
    - The core operation of this function is to **build a highly customized prompt**. It doesn't simply ask the model to describe the image. Instead, it embeds `object_name` within a carefully designed template, explicitly instructing the model to "locate each {object_name} in the image," and implicitly or explicitly requests that the results be returned in a specific format (such as an array of coordinates).
    - After constructing the prompt, it calls the `infer_with_image` method of the model interface layer, passing the image and this customized instruction.
    - After receiving the text returned from the model interface layer, it also performs **post-processing**: using methods such as regular expressions to parse the model's natural language response into precise coordinate data.
    - Finally, it returns the parsed structured coordinate data to the upper-layer application.
2. **Model interface layer (`large_model_interface.py`)**:
    - The `infer_with_image` function still serves as the "dispatching center." It receives the image and prompt from `visual_positioning` and dispatches the task to the correct backend model implementation based on the current configuration (`self.llm_platform`).
    - For visual positioning tasks, the model interface layer's responsibilities are essentially the same as for visual understanding tasks: correctly packaging image data and text instructions, sending them to the selected model platform, and then returning the returned text results intact to the tool layer. All platform-specific implementation details are encapsulated in this layer.

In summary, the general workflow for visual localization is: ToolsManager receives the target object name and constructs a precise prompt requesting coordinates. ToolsManager calls the model interface. ModelInterface packages the image and prompt together and sends them to the corresponding model platform according to the configuration. The model returns text containing

the coordinates. ModelInterface returns this text to ToolsManager. ToolsManager parses the text, extracts the structured coordinate data, and returns it. This process demonstrates how Prompt Engineering can enable a general-purpose large visual model to accomplish more specific and structured tasks.

3. Practical Operations

3.1 Configuring the Offline Large Model

3.1.1 Configuring the LLM Platform (yahboom.yaml)

This file determines which large model platform the model_service node loads as its primary language model.

1. **Open the file in terminal**:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. **Modify/confirm** `llm_platform`:

```
model_service:                          #模型服务器节点参数 Model server node parameters
  ros__parameters:
    language: 'en'                      #大模型接口语言 Large Model Interface Language
    useolinetts: False                   #文字模式下此项无效，可忽略 This item is invalid in text mode and can be ignored

    # 大模型配置 Large model configuration
    llm_platform: 'ollama'              # 关键: 确保这里是 'ollama' Key: Make sure it's 'ollama'
    regional_setting : "international"
```

3.1.2 Configuring the Model Interface (`large_model_interface.yaml`)

This file defines which vision model to use when the `ollama` platform is selected.

1.Open the file in Terminal

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2.Find the ollama related configuration

```
#.....
## 离线大模型 (Offline Large Language Models)
# Ollama配置 Ollama Configuration
ollama_host: "http://localhost:11434"  # Ollama服务器地址 Ollama server address
ollama_model: "llava"            # 关键: 将这里改为你已下载的多模态模型，如 "llava" Key: Change this to the multimodal model you downloaded, such as "llava"
#.....
```

Note: Please make sure that the model specified in the configuration parameters (such as `llava`) can handle multimodal input.

If you have modified any configuration files, please run the following command to compile the ROS workspace for the configuration to take effect.

```
cd ~/yahboom_ws
colcon build
```

3.2 Starting and Testing the Feature (Text Mode)

1. **Prepare image files**:

   Place an image file to test in the following path: /home/jetson/yahboom_ws/src/largemodel/resources_file/visual_positioning

   Then name the image `test_image.jpg`

2. **Start the `largemodel` main program (in text mode):**

   Open a terminal and run the following command：

   ```
   ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
   ```

3. **Send text command**:
   Open another terminal again and run the following command,

   ```
   ros2 run text_chat text_chat
   ```

   Then start typing text: "Analyze the position of the dinosaur in the picture."

   If the program does not use the default image, pass in the image path or name, for example, test_image.jpg.

4. **Observations**:In the first terminal where you run the main program, you'll see log output indicating that the system received the command, called the `visual_positioning` tool, completed the execution, and saved the coordinates to a file.

   This file can be found in the `~/yahboom_ws/src/largemodel/resources_file/visual_positioning` directory.

---

4. Common Problems and Solutions

Problem 1: "Image file not found" error message

Solution:

1. **Check the path**: Make sure you have placed the image file in the specified path, named it `test_image.jpg`, and have read permissions for the file.
2. **Image format**: Make sure the image file is not corrupted and is in .jpg format.